

Multi-Agent Mentor-Student Simulation — Assignment Solution

Margarita Benedichuk

2026-07-11

Assignment. Two LLM agents run a 10-lesson “Prompt Engineering” course: a **mentor** checks *application* of each skill (not recall), using questions only — no files/screenshots shown to the student. The **student** sometimes bluffs (claims practice it didn’t do). The mentor must catch this from conversation alone.

Architecture

- **Two mirrored contexts, not one shared chat** — mentor_ctx/student_ctx are separate message lists; the relay forwards each turn into the other’s context. Fresh per lesson; cross-lesson memory rides durable tools (mentor_ledger, student_practice_log), not raw chat history.
- **Fixed-phase flow + forced gate:** open (explain + verify) → apply (transfer scenario) → probe (adaptive follow-ups) → forced advance_decision. Guarantees one structured verdict (PASS/BLUFF_SUSPECTED) per lesson.
- **Bluff detection, three levers:** (1) *canary facts* — every rule uses a made-up term (“1-task-1-verb rule”, “Contrast rule”, ...) absent from any model’s pretraining; (2) answer_key.json — private to the mentor, canary facts + expected application points; (3) student_practice_log.json — on bluff lessons (3, 6, 9) nothing is written, so practice_read returns null and the student must fabricate or admit it.

Models & tools

Mentor/student both deepseek/deepseek-v4-pro (OpenRouter); student runs with reasoning disabled (kills empty-message blank-outs). Judge is a deterministic precision/recall calculator (no LLM). Tools: ledger_read/ledger_write (mentor memory), practice_read (student’s only source of concrete detail), practice_simulator (deterministic, seeded practice-outcome generator), advance_decision (the hard gate), judge (post-run scoring).

Result (best run — run_012, 10/10 lessons, seed 7)

```
precision=1.0  recall=0.667  TP=2 FP=0 FN=1
caught=[6, 9]  missed=[3]    (bluffs forced on 3, 6, 9)
```

Lessons 3, 6, 9 were the only ones the mentor probed to the full budget (4/4 rounds) before deciding — it never accepted a first answer on exactly the forced-bluff lessons.

Reflection

Lesson 3’s bluff slipped through: the student fabricated a plausible, internally consistent practice story (a support-ticket classifier, a “too subtle” counter-example, an admitted “I don’t recall the exact text”) — friction specific enough to read as real. That’s the honest ceiling of conversation-only detection: a strong model can manufacture *plausible* friction, not just a smooth denial, and nothing here can tell manufactured friction from real friction once both are equally specific. Mentor prompt went through 4 iterations (early versions accepted a correct transfer answer alone, or condemned a thin-but-honest first answer too soon); student went through 3 (mainly hardening “specifics only from your practice log”).

Full detail in the repository

- Mentor / student prompts: `prompts/mentor/current.txt`, `prompts/student/current.txt`
- Tools implementation: `orchestration/tools.py`, `orchestration/relay.py`
- Full best-run transcript: `courses/prompt-engineering/logs/run_012/transcript.txt`
- QA across all runs: `courses/prompt-engineering/logs/QA_REPORT.md`
- Full 5-section writeup: `docs/final_submission.md`